

Figure 1 Solution Revision Following Later-Stated Constraints (Motivating Case Study)

DevGPT: dba

1. Initial Prompt & Solution Anchor

TURN 1 (D0001 → A0001)



2. Later-Stated Constraints (LSRI)

TURN 2 (D0002)



3. Solution Revision & Elicitation



Developer

Turn D0001

"how to implement filter by date option (default is this month's expenses) in ruby on rails?"
[Attached partial view _expenses_table.html.erb]



ChatGPT

Solution Anchor

```
// [Date range calculation omitted]
def index
  @expenses = Expense.where(datetime: start_date..end_date)
end
```



Unconstrained Global Solution:

Directly queries all Expense records without scoping to the logged-in user, unauthenticated handling, or ordering.



Developer

Developer-initiated

"here is expenses_controller"

```
def index
  @expenses = user_signed_in? ?
    current_user.expenses.order(...) : Expense.none
end
```

U1 User-Specific Scope

Actor & Perm

EEO=Yes

Revised

Restrict query to current_user.expenses (absent in Turn 1).

U2 Signed-Out Behavior

Actor & Perm

EEO=Yes

Revised

Return empty relation Expense.none when not signed in.

U3 Descending Ordering

Functional

EEO=Yes

Extended

Preserve order: order(datetime: :desc, created_at: :desc).



Earlier Elicitation Opportunity (Before A0001)

"Whose expenses should be listed (current user only?), and what if the user is unauthenticated?"



ChatGPT

```
// [Date range calculation omitted]
def index
  if user_signed_in?
    @expenses = current_user.expenses
      .where(datetime: start_date..end_date)
      .order(datetime: :desc)
  else
    @expenses = Expense.none
  end
end
```

- U1 (Revised): Scope replacement
- U2 (Revised): Added authentication
- U3 (Extended): Re-integrated

Takeaway: Proactively eliciting constraints from naive solution (A0001) and avoiding